Scalable Malware Retrieval: Sub-linear Function Call Graph Matching via Embeddings and Locality Sensitive Hashing

Aayush Kuloor
Roll No: 24110004
aayush.kuloor@iitgn.ac.in
IIT Gandhinagar

Dhru Harsh Darshakbhai
Roll No: 24110107
harsh.dhru@iitgn.ac.in
IIT Gandhinagar

Laxmidhar Panda
Roll No: 24110185
24110185@iitgn.ac.in
IIT Gandhinagar

Raj Amit Modi
Roll No: 24110282
raj.modi@iitgn.ac.in
IIT Gandhinagar

Abstract

We study retrieval over Android malware function call graphs (FCGs). Rather than assigning a single closed-set label to a query graph, we retrieve the most similar graphs from a labeled corpus and use the ranked neighbors for family inference. We evaluate the full pipeline on **MalNet-Tiny**, moving from a simple baseline to a stronger representation and search framework.

Our baseline combines Graph2Vec embeddings with locality-sensitive hashing (LSH). We then replace that pipeline with a graph neural encoder based on a Graph Isomorphism Network (GIN), supervised contrastive training, and FAISS HNSW indexing over normalized graph embeddings. We also study metric-aligned hyperparameter search, family-balanced sampling, Proxy-Anchor loss, self-supervised pretraining through GraphCL and GraphMAE, hard-negative mining, and an optional graph matching reranker.

Our main empirical finding is that retrieval quality depends primarily on the quality of the learned graph representation, whereas latency is driven mostly by the nearest-neighbor index. Our final pipeline is both accurate and fast, and it provides a clean retrieval benchmark on MalNet-Tiny. More broadly, our results show how feature design, graph representation learning, metric-aware training, and scalable nearest-neighbor search work together in a single malware retrieval pipeline.

Keywords

malware retrieval, function call graphs, graph neural networks, supervised contrastive learning, approximate nearest neighbor search, MalNet-Tiny

1 Introduction

Malware analysis often benefits more from retrieval than from ordinary classification. For a new program, an analyst rarely wants only a hard label; they also want to know which known samples it most closely resembles, whether those neighbors belong to a coherent family, and what structural behavior they share. Retrieval addresses that need directly by ranking a corpus of known samples against the query.

Function call graphs are a natural representation for this setting because they capture inter-procedural structure and are more robust than raw byte signatures to renaming, packing, and other superficial code transformations. In this work, we build a retrieval pipeline on top of MalNet [1], using **MalNet-Tiny** as our benchmark. We trace the progression from a weak baseline to a much stronger representation-and-search pipeline, and show which design choices matter most.

2 Problem Setup and Data

Let the searchable corpus be

$$C = \{(G_i, y_i)\}_{i=1}^{N},$$

where each $G_i = (V_i, E_i, X_i)$ is a directed FCG and y_i is its malware family label. For a held-out query graph G_q, the system returns a ranked list

$$\pi_q(1), \pi_q(2), \ldots, \pi_q(k),$$

ordered by embedding similarity. The retrieval-derived family prediction is the majority label among the top-k neighbors.

Dataset scope. We use **MalNet-Tiny** throughout. Our evaluation setup supports both explicit split files and automatic split generation. In the standard pipeline, we create family-stratified 70/15/15 splits with seed 42. For retrieval, we use the train split as the search corpus and the test split as the query set.

Preprocessing pipeline. Each raw sample is stored as an edge list and is first converted into a directed NetworkX graph before being mapped to a PyTorch Geometric object. During this conversion, nodes are re-indexed, edge tensors are constructed, and self-loops are added so that isolated or weakly connected nodes still participate in message passing. This is not just a file-format conversion; it defines the learning object on which both the baseline and the neural models operate.

Node features. Each graph is converted into a PyTorch Geometric `Data` object with self-loops and a 22-dimensional node feature vector. The feature blocks are:

- 10-dimensional API-category one-hot features,
- log-scaled in-degree and out-degree,
- entry-node and exit-node indicators,
- a hashed node-name bucket feature,
- seven structural features: PageRank, reverse PageRank, k-core, strongly connected component size, distance from entry, distance to exit, and clustering coefficient.

An important detail is that many MalNet node identifiers are numeric strings rather than semantic API names. As a result, a large

fraction of nodes map to the coarse `other` bucket, which makes structural features especially important. This is one of the main reasons we place substantial emphasis on feature design rather than treating model choice alone as the solution.

Why feature engineering matters here. If node names carried strong semantic information, a coarse API taxonomy would already separate many malware behaviors. In MalNet-Tiny, however, much of that signal is absent or weak. The final node representation therefore combines local structural cues such as degree and entry/exit roles with global cues such as PageRank, strongly connected component size, and graph-position features from BFS distances. This allows the encoder to reason about both the functional role of a node and its place within the broader call graph.

3 From Baseline to Final Framework

3.1 Baseline: Graph2Vec + LSH

The starting point is a simple retrieval pipeline built from Graph2Vec [10] and random-projection LSH [2]. Graph2Vec treats each graph as a multiset of rooted Weisfeiler-Lehman subgraph hashes and uses a Doc2Vec-style model [7] to learn a fixed-length graph embedding:

$$G \mapsto D_G \mapsto z_G.$$

In our setup, these embeddings are 128-dimensional.

LSH provides an inexpensive approximate nearest-neighbor baseline by hashing embeddings through random hyperplanes and reranking bucket collisions using exact cosine similarity. This gives us a useful starting point: it supports end-to-end graph retrieval without task-specific neural message passing or advanced indexing. It also serves as a diagnostic baseline. If retrieval is weak even with approximate search in place, the main bottleneck is likely the quality of the representation rather than the search algorithm alone.

3.2 Representation upgrade: GIN encoder

The main model replaces Graph2Vec with a 5-layer Graph Isomorphism Network (GIN) [14], hidden dimension 256, dropout 0.3, and a two-layer projection head. At layer ℓ,

$$h_v^{(\ell+1)} = \mathrm{MLP}^{(\ell)}\left(h_v^{(\ell)} + \sum_{u \in \mathcal{N}(v)} h_u^{(\ell)}\right).$$

After message passing, the graph representation concatenates global sum pooling and global mean pooling:

$$g = \left[\sum_{v \in V} h_v^{(L)} \,\middle\|\, \frac{1}{|V|}\sum_{v \in V} h_v^{(L)}\right].$$

The projection head maps g to a 256-dimensional L2-normalized embedding $z \in \mathbb{R}^{256}$.

This change matters because it moves us from an unsupervised graph-level embedding heuristic to a learned message-passing encoder that can use both local neighborhood structure and global graph statistics. In our ablations, this is the dominant source of retrieval-quality improvement.

Why GIN is a good fit for FCGs. GIN is a strong fit here because it is designed to learn expressive graph representations under sum aggregation. In our setting, that translates into a practical advantage: the model can capture whether a node belongs to a dense routine, a sparse branch, a shared hub, or a terminal pathway, and then fold that information into a graph-level representation. The combined sum and mean pooling step is also deliberate. Sum pooling preserves size-sensitive evidence, while mean pooling captures average structural behavior, so the encoder can use both magnitude and normalized pattern statistics.

3.3 Training objectives: CE and SupCon

To isolate the effect of the training objective, we study both cross-entropy and supervised contrastive training. The supervised contrastive objective [5] is

$$\mathcal{L}_{\mathrm{SupCon}} = -\frac{1}{|I|}\sum_{i \in I}\frac{1}{|P(i)|}\sum_{p \in P(i)}\log\frac{\exp(z_i^\top z_p/\tau)}{\sum_{a \neq i}\exp(z_i^\top z_a/\tau)},$$

with temperature $\tau = 0.07$ in the main configuration.

The training loop uses two stochastic graph views generated by edge dropout and feature masking. Cross-entropy remains a useful control because it tells us whether the encoder itself is already strong enough to support retrieval. SupCon is more closely aligned with retrieval because it explicitly shapes the embedding geometry rather than optimizing only classification logits.

Optimization details. We train the encoder with AdamW, linear warmup followed by cosine decay, gradient clipping, and early stopping on validation MAP@10. These choices matter for retrieval because they reduce the chance that the model looks strong on training labels while drifting away from the ranking geometry needed at test time. We also tune hyperparameters directly against MAP@10 rather than relying on an indirect surrogate.

Why CE remains in the study. Including a cross-entropy variant is methodologically useful. It separates the effect of the encoder architecture from the effect of the retrieval objective and shows whether the gains come from GIN itself or from the contrastive loss. That makes the ablations far more interpretable than a single final-model comparison.

3.4 Search upgrade: HNSW

The stronger retrieval backend is FAISS HNSW [4, 9]. Since embeddings are unit-normalized,

$$\|x - y\|_2^2 = 2 - 2\cos(x, y),$$

so nearest-neighbor search in L2 is equivalent to cosine search. The main configuration uses $M = 32$, `efConstruction=200`, and `efSearch=50`.

This change is conceptually distinct from the encoder upgrade. GIN improves the quality of the representation space, whereas HNSW improves how efficiently we search that space. Our ablations are designed to separate those two effects.

Why HNSW improves the pipeline. The baseline LSH backend discretizes the embedding space through hash buckets, which is simple but coarse once the representation becomes strong and highly structured. HNSW instead builds a hierarchical proximity graph

over the embeddings and supports rapid multi-scale navigation. In practical terms, the search backend becomes much better at exploiting the geometry produced by the encoder. This is why the overall improvement naturally falls into two stages: first we make the embedding space meaningful, and then we make search in that space efficient.

3.5 Extended pipeline components

Beyond the core ablation, our full pipeline includes several additional components:

- **MAP@10-aligned HPO:** Optuna tuning is aligned with the retrieval metric we care about most.
- **Family-balanced sampling:** a custom batch sampler ensures that contrastive training sees meaningful in-batch positives even on long-tailed families.
- **Proxy-Anchor loss:** class proxies provide a stable metric learning alternative when batch-local positives are sparse [6].
- **GraphCL and GraphMAE pretraining:** self-supervised pretraining improves initialization through augmented-view contrastive learning [15] and masked feature reconstruction [3].
- **Hard-negative mining and GMN reranking:** the first-stage ANN retriever produces informative confusing pairs, and an optional graph matching network refines the top shortlist with cross-graph interactions [8].

Taken together, these components give us a more complete retrieval pipeline, with explicit mechanisms for representation learning, class-imbalance handling, self-supervision, and second-stage reranking.

Balanced sampling and Proxy-Anchor loss. Malware family distributions are highly imbalanced. In a batch-local contrastive objective, that can leave rare families underrepresented and weaken the positive-pair signal. Our custom `FamilyBalancedSampler` addresses this by ensuring that each batch contains multiple samples from selected families. Proxy-Anchor loss complements that idea by introducing class-level proxy vectors in the embedding space, making training less sensitive to the exact composition of each mini-batch.

Self-supervised pretraining. GraphCL and GraphMAE address a different issue: initialization. GraphCL encourages invariance across augmented graph views, whereas GraphMAE forces the encoder to reconstruct masked feature information from graph context. Together, these methods give us a path toward stronger structure learning even before we apply label supervision.

Hard-negative mining and reranking. The final extension is a two-stage retrieval design. First, the encoder and ANN index produce a shortlist. Then an optional graph matching reranker compares the query against those shortlisted graphs more directly through cross-graph interactions. Hard-negative mining is crucial here: instead of training the reranker on arbitrary random pairs, we focus on the cases where the first-stage retriever is genuinely confused.

4 Experimental Design

We use PyTorch Geometric for graph handling and GNN training, and FAISS for approximate nearest-neighbor search. Our ablation design varies the embedding model and retrieval backend separately so that we can analyze representation quality and indexing quality independently.

We ask five main experimental questions:

- How weak is the original Graph2Vec + LSH baseline?
- How much does GIN help if the retrieval index remains unchanged?
- Does supervised contrastive training improve the embedding space beyond cross-entropy?
- How much does HNSW help when the embedding is weak?
- What is the strongest overall design when both the encoder and the retrieval backend are upgraded?

Training and indexing configuration. Our main encoder uses five GIN layers, hidden dimension 256, embedding dimension 256, dropout 0.3, augmentation probabilities of 0.1 for edge dropout and feature masking, and early stopping on validation MAP@10. Our main HNSW index uses $M = 32$, `efConstruction`=200, and `efSearch`=50. These settings reflect the balance we aim for between representation quality, retrieval latency, and training stability.

Evaluation protocol. For a query q, let $rel_q(r) = 1$ if the item at rank r has the same family label as the query and 0 otherwise. If n_q is the number of relevant items in the corpus, then

$$\mathrm{AP@}k(q) = \frac{1}{\min(n_q, k)} \sum_{r=1}^{k} P_q(r)\, rel_q(r), \qquad P_q(r) = \frac{1}{r} \sum_{t=1}^{r} rel_q(t).$$

Mean Average Precision is

$$\mathrm{MAP@}k = \frac{1}{|Q|} \sum_{q \in Q} \mathrm{AP@}k(q).$$

We also report Recall@k, MRR, and retrieval-derived Accuracy@10 (majority vote over the top-10 labels). `Latency_ms` is the mean single-query time measured during retrieval.

Why these metrics are reported together. MAP@k is our primary retrieval-quality metric because it rewards placing relevant results high in the ranking. Recall@k captures coverage of relevant neighbors in the shortlist, MRR captures how early the first correct result is found, and Accuracy@10 provides a simple downstream family-prediction view through majority vote. Reporting all four gives a more complete picture than any single metric alone.

5 Results and Discussion

Table 1 summarizes the quantitative results of our main ablation study. We draw three robust conclusions from these findings.

First, the largest quality improvement comes from replacing Graph2Vec with GIN. Once the encoder becomes strong, both CE and SupCon variants outperform the Graph2Vec baseline by a wide margin. This indicates that the main bottleneck in the baseline pipeline is the learned representation rather than the search index.

Second, HNSW is the key latency improvement. Moving from LSH to HNSW changes the practical usability of the system because

Experiment	MAP@1	MAP@5	MAP@10	MAP@50	R@1	R@5	R@10	R@50	MRR	Acc@10	Latency (ms)
Graph2Vec + LSH	0.2000	0.1430	0.1152	0.0589	0.2000	0.1856	0.1723	0.1218	0.2979	0.2120	2.2847
GIN-CE + LSH	0.9060	0.8874	0.8705	0.8659	0.9060	0.9024	0.8993	0.8888	0.9249	0.9160	3.3238
GIN-SupCon + LSH	0.9000	0.8821	0.8779	0.8701	0.9000	0.8970	0.8964	0.8942	0.9202	0.9110	3.7413
Graph2Vec + HNSW	0.1990	0.1708	0.1548	0.1257	0.1990	0.1928	0.1890	0.1927	0.2528	0.1910	0.0927
GIN-SupCon + HNSW	**0.9080**	**0.8902**	**0.8880**	**0.8708**	**0.9080**	**0.9151**	**0.9019**	**0.8949**	**0.9271**	**0.9219**	**0.0808**

Table 1: Main retrieval ablation on MalNet-Tiny

it delivers a much faster search stage without changing the surrounding evaluation protocol. Figures 1 and 2 make this distinction clear by separating the quality and latency stories.

Third, we view the final system as the strongest overall quality-speed trade-off rather than as a single-number optimization. In the HNSW setting, the SupCon-trained encoder is the stronger retrieval model, which is why we take **GIN-SupCon + HNSW** as our preferred end-to-end framework. The more careful claim is not that one loss wins on every metric in every setting, but that the contrastive HNSW pipeline is the most effective overall design for the retrieval task we study.

Figure 3 complements the table by showing how ranking quality behaves across retrieval depths. This matters because malware retrieval is not only about the first few returned graphs: the shape of the full MAP@k curve shows how well the representation preserves family structure deeper in the candidate shortlist.

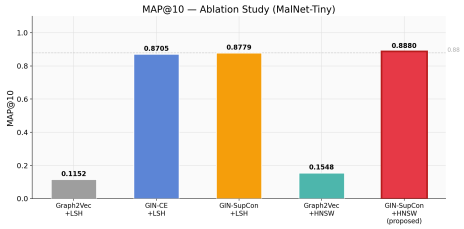


Figure 1: MAP@10 comparison across systems. The largest quality improvement comes from replacing the baseline graph representation with the GIN-based encoder.

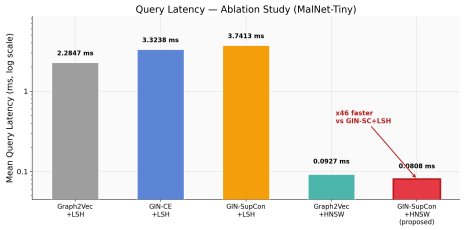


Figure 2: Latency comparison across systems. The search backend upgrade from LSH to HNSW is the main reason our final pipeline achieves much faster retrieval.

The same point appears in a joint quality-latency view. As shown in Figure 4, Graph2Vec + HNSW remains weak despite using the faster index, which shows that a better search method cannot rescue

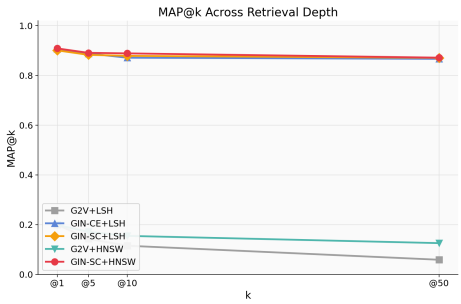


Figure 3: MAP@k curves across retrieval depth. The GIN-based systems maintain strong ranking quality deeper into the shortlist, whereas the baseline systems degrade much earlier.

a poor embedding geometry. Conversely, the GIN-based systems remain strong even with the weaker LSH backend, which confirms that representation learning is the dominant factor behind retrieval accuracy.

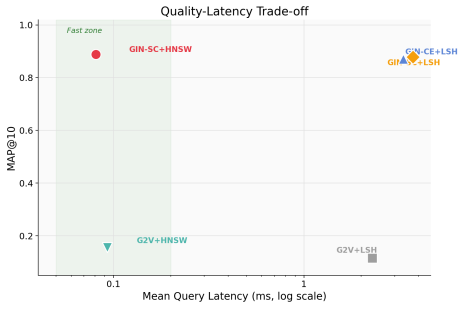


Figure 4: Quality-latency trade-off. The proposed framework occupies the most favorable region by combining a strong encoder with a fast ANN index.

6 Relation to Prior Work and Limitations

We did not find a published benchmark that reports directly comparable MalNet-Tiny retrieval metrics such as MAP@10 or MRR. The original MalNet work is a graph classification benchmark [1], and the BCG paper is also classification-focused [11]. So direct numerical comparison against prior MalNet retrieval baselines is not currently justified.

There are still useful methodological neighbors. Retrieval-style binary similarity work such as jTrans [13] and IRBinDiff [12] supports the broader idea that program-structure-aware representation learning is effective, even though those papers operate on different datasets and problem formulations.

Our study should also be read with two limitations in mind. First, the current evaluation is in-benchmark and family-stratified, so it does not by itself resolve out-of-distribution robustness. Second, the BCG paper argues that MalNet-Tiny contains substantial duplication [11]; that caveat matters for retrieval as well because near-duplicates can inflate nearest neighbor metrics. For that reason, we view our results as a strong benchmark outcome and a well-supported pipeline design, not as a final statement about real-world deployment performance.

7 Conclusion

We develop an end-to-end retrieval pipeline for Android malware function call graphs and trace its evolution from a simple baseline to a much stronger final framework. Graph2Vec + LSH provides a useful reference point, but it is limited by weak graph representations. Replacing Graph2Vec with GIN and richer node features gives the largest retrieval-quality gain. Replacing LSH with HNSW gives the largest latency gain. Together, these changes yield a system that is accurate, fast, and well supported by our ablation design.

Beyond a single optimized model, our full system also integrates metric-aligned tuning, imbalance-aware training, self-supervised pretraining, hard-negative mining, and optional second-stage reranking. As a result, we see this work not only as a strong experimental baseline, but also as a coherent and extensible retrieval framework for future Android malware research.

References

[1] Scott Freitas, Yuxiao Dong, Joshua Neil, and Duen Horng Chau. 2021. A Large-Scale Database for Graph Representation Learning. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.

[2] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *Proceedings of the 25th International Conference on Very Large Data Bases*. 518–529.

[3] Zhenyu Hou, Xiao Wang, Chuan Shi, Hao Liu, Yanfang Shen, and Xunqiang Jiang. 2022. GraphMAE: Self-Supervised Masked Graph Autoencoders. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 594–604.

[4] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.

[5] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. 2020. Supervised Contrastive Learning. In *Advances in Neural Information Processing Systems*, Vol. 33. 18661–18673.

[6] Sungyeon Kim, Dongwon Kim, Minsu Cho, and Suha Kwak. 2020. Proxy Anchor Loss for Deep Metric Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 3238–3247.

[7] Quoc Le and Tomas Mikolov. 2014. Distributed Representations of Sentences and Documents. In *Proceedings of the 31st International Conference on Machine Learning*. 1188–1196.

[8] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. 2019. Graph Matching Networks for Learning the Similarity of Graph Structured Objects. In *Proceedings of the 36th International Conference on Machine Learning*. 3835–3845.

[9] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[10] Annamalai Narayanan, Mahinthan Chandramohan, Lihui Chen, Yang Liu, and Sanjay Jha. 2017. graph2vec: Learning Distributed Representations of Graphs. *arXiv preprint arXiv:1707.05005* (2017).

[11] Erdem Raff, James Holt, Sarah M. Erfani, Richard N. Zare, and Charles Nicholas. 2025. Better Call Graphs: A New Dataset of Function Call Graphs for Malware Classification. *arXiv preprint arXiv:2512.20872* (2025).

[12] Xiuwei Shang, Yufei Li, Chenxiao Wu, Yao Cheng, and Kui Ren. 2024. Binary Code Similarity Detection via Graph Contrastive Learning on Intermediate Representations. *arXiv preprint arXiv:2410.18561* (2024).

[13] Wenjie Wang, Hao Sun, Qiuping Wang, Xiang Gao, and Yang Liu. 2022. jTrans: Jump-Aware Transformer for Binary Code Similarity Detection. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1–13.

[14] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *International Conference on Learning Representations*.

[15] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph Contrastive Learning with Augmentations. In *Advances in Neural Information Processing Systems*, Vol. 33. 5812–5823.